# NBA Edge Model

Complete Walkthrough: Phase 0 through Phase 2

Every literal step from opening your laptop to having a trained ML model that outputs player prop probabilities. Nothing is assumed. Every command, every click, every file.

**Time estimate: ~15-20 hours total across all parts**

How to use this: Go in order. Do not skip parts. Every checkpoint must pass before you move on.

**TABLE OF CONTENTS**

PART 0: Machine Setup (~30 min)

PART 1: Project Setup (~20 min)

PART 2: Data Ingestion (~3-6 hours, mostly waiting)

PART 3: Data Exploration / EDA (~2-3 hours)

PART 4: Feature Engineering (~4-5 hours)

PART 5: Model Training (~3-4 hours)

PART 6: Edge Finding and Backtesting (~2-3 hours)

# PART 0: MACHINE SETUP

**Goal:** Get Python, VS Code, and Git installed and working on your Mac.

## Step 0.1 — Open Terminal

Press Cmd + Space on your Mac. Type 'Terminal'. Press Enter.

You will see a window with a blinking cursor. This is where you type commands. Every command in this guide that appears in a grey box means: type it into Terminal and press Enter.

## Step 0.2 — Install Homebrew

Homebrew is a tool that installs other tools. Paste this entire line into Terminal:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It will ask for your Mac password (the one you use to log in). **You will not see characters as you type** — that is normal. Type it and press Enter.

When it finishes, it may print instructions that say 'Add Homebrew to your PATH' with two commands. **Run those commands.** They look something like:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' &gt;&gt; ~/.zprofile eval
"$(/opt/homebrew/bin/brew shellenv)"
```

Close Terminal completely (Cmd + Q), then reopen it. Verify it works:

```
brew --version
```

You should see something like Homebrew 4.x.x. If you see 'command not found,' run the PATH commands above again.

## Step 0.3 — Install Python

```
brew install python@3.12
```

Wait for it to finish. Then verify:

```
python3 --version
```

You should see Python 3.12.x (the exact number after 12 does not matter).

**If you see Python 3.9 or similar:** That is the old Mac system Python. Run:

```
brew link python@3.12 --force
```

Then close and reopen Terminal and try again.

## Step 0.4 — Install Git

```
brew install git
```

Then configure it with your real name and email:

```
git config --global user.name "Your Name" git config --global user.email
"your.email@berkeley.edu"
```

Verify:

```
git --version
```

## Step 0.5 — Install VS Code

1. Open Safari or Chrome

2. Go to: https://code.visualstudio.com

3. Click the big blue download button

4. Open the downloaded .zip file — it will extract an app

5. Drag Visual Studio Code.app into your Applications folder

6. Open it from Applications

Now install the command-line shortcut so you can open VS Code from Terminal:

1. In VS Code, press Cmd + Shift + P

2. Type: shell command

3. Click: Shell Command: Install 'code' command in PATH

Now install extensions. Click the square icon on the left sidebar (Extensions). Search for and install each of these:

- **Python** (by Microsoft)

- **Jupyter** (by Microsoft)

- **SQLite Viewer** (by alexcvzz)

## CHECKPOINT

- [ ] brew --version shows a version number
- [ ] python3 --version shows Python 3.12.x
- [ ] git --version shows a version number
- [ ] code --version shows a version number

# PART 1: PROJECT SETUP

**Goal:** Unpack the project, install dependencies, and verify the data pipeline works.

## Step 1.1 — Create the project folder

```
mkdir -p ~/projects cd ~/projects
```

## Step 1.2 — Unzip the project

Find the nba-edge-model.zip you downloaded from our conversation. It is probably in ~/Downloads. Run:

```
cp ~/Downloads/nba-edge-model.zip ~/projects/ cd ~/projects unzip nba-edge-model.zip mv
nba_model_project nba-edge-model cd nba-edge-model
```

Verify the structure:

```
ls
```

You should see: ARCHITECTURE.md, README.md, config, data, requirements.txt, scripts, src, etc.

## Step 1.3 — Create a virtual environment

A virtual environment is an isolated box for this project's packages. Without it, packages from different projects can conflict.

```
python3 -m venv venv
```

This creates a venv/ folder. Now activate it:

```
source venv/bin/activate
```

**You should now see (venv) at the start of your terminal prompt.** This means you are inside the virtual environment.

**IMPORTANT:** Every time you open a new Terminal window to work on this project, you must run:

```
cd ~/projects/nba-edge-model &amp;&amp; source venv/bin/activate
```

To make this easier, add a shortcut:

```
echo 'alias nba="cd ~/projects/nba-edge-model &amp;&amp; source venv/bin/activate"'
&gt;&gt; ~/.zshrc source ~/.zshrc
```

Now you can just type **nba** to jump into the project.

## Step 1.4 — Install Python packages

Make sure (venv) is showing in your prompt, then:

```
pip install --upgrade pip pip install -r requirements.txt
```

This downloads and installs ~20 packages. Takes 1-3 minutes.

## Step 1.5 — Open the project in VS Code

```
code .
```

VS Code opens with your project in the sidebar. Now tell VS Code to use your virtual environment:

1. Press Cmd + Shift + P

2. Type: Python: Select Interpreter

3. Choose the one that says ./venv/bin/python (it should have 'venv' in the path)

## Step 1.6 — Initialize the database

```
python -c "from src.utils.db import init_db; init_db()"
```

You should see: Database initialized at .../data/nba_edge.db

## Step 1.7 — Test the pipeline

This pulls one player's data to prove everything works:

```
python scripts/pull_season_data.py --test
```

You should see Jokic's stats print out and a test complete message.

**Troubleshooting:**

- ModuleNotFoundError: No module named 'nba_api' — You are not in the venv. Run: source venv/bin/activate

- ConnectionError or timeout — NBA.com might be down. Wait 5 min and try again.

- No data returned — The NBA season might be over. The backfill will still pull historical data.

## Step 1.8 — Initialize Git

```
git init git add . git commit -m "initial project scaffold"
```

## Step 1.9 — Create GitHub repo

1. Go to https://github.com/new

2. Name: nba-edge-model

3. Description: Quantitative NBA prediction model

4. Set to **Private** (make public later when polished)

5. Do NOT check any initialization boxes

6. Click Create repository

7. Back in Terminal:

```
git remote add origin https://github.com/YOUR_USERNAME/nba-edge-model.git git branch -M
main git push -u origin main
```

If it asks for a password, you need a Personal Access Token instead: go to https://github.com/settings/tokens, generate a classic token with 'repo' scope, and use that as your password.

## CHECKPOINT

- [ ] (venv) shows in your terminal prompt
- [ ] python scripts/pull_season_data.py --test prints Jokic's stats
- [ ] data/nba_edge.db exists (check in VS Code sidebar under data/)
- [ ] Your GitHub repo shows the project files

# PART 2: DATA INGESTION

**Goal:** Fill the database with 5+ seasons of player game logs.

## Step 2.1 — Pull the current season first

This is faster than the full backfill and gives you data to explore while the rest loads:

```
python scripts/pull_season_data.py --season 2024-25
```

**This takes 30-60 minutes.** The NBA API allows roughly 1 request per 0.6 seconds, and there are 500+ players. You will see a progress bar. Let it run.

## Step 2.2 — Verify current season data

When it finishes, run this verification command:

```
python3 -c " from src.utils.db import get_connection import pandas as pd conn = get_connection() count = pd.read_sql('SELECT COUNT(*) as n FROM game_logs', conn).iloc[0]['n'] players = pd.read_sql('SELECT COUNT(DISTINCT player_id) as n FROM game_logs', conn).iloc[0]['n'] print(f'Total game log rows: {count:,}') print(f'Unique players: {players:,}') conn.close() "
```

You should see ~25,000-40,000 rows and 400-500 players.

## Step 2.3 — Backfill historical seasons

Now pull the remaining seasons. **This takes several hours.** Run it and leave your laptop open:

```
python scripts/pull_season_data.py --backfill
```

> *Tip: Prevent your laptop from sleeping. Go to System Settings > Energy and set display sleep to Never while this runs.*
>
> *If it crashes partway through: Just run the same command again. The pipeline tracks what it already pulled and skips those players. Safe to restart.*

## Step 2.4 — Verify full dataset

After backfill completes:

```
python3 -c " from src.utils.db import get_connection import pandas as pd conn = get_connection() summary = pd.read_sql(''' SELECT season, COUNT(*) as games, COUNT(DISTINCT player_id) as players FROM game_logs GROUP BY season ORDER BY season ''', conn) print(summary.to_string(index=False)) total = pd.read_sql('SELECT COUNT(*) as n FROM game_logs', conn).iloc[0]['n'] print(f'Total rows: {total:,}') conn.close() "
```

You should see 6 seasons, totaling 150,000-250,000 rows.

## Step 2.5 — Commit

```
git add . git commit -m "phase 1: data ingestion complete" git push
```

### CHECKPOINT

[ ] Every season shows data in the summary query

[ ] Total rows > 100,000

[ ] No seasons are missing or suspiciously small

[ ] Changes pushed to GitHub

# PART 3: DATA EXPLORATION (EDA)

**Goal:** Understand what the data looks like before engineering features from it.

**What you will learn:** Jupyter notebooks, pandas data exploration, matplotlib visualization.

## Step 3.1 — Start Jupyter

In Terminal (with (venv) active):

```
jupyter notebook
```

This opens a browser window showing your project files. Click notebooks/, then click New > Python 3 in the top right.

Rename the notebook: click 'Untitled' at the top and type 01_data_exploration.

A notebook has **cells**. Each cell holds code. Press **Shift + Enter** to run a cell and move to the next one.

## Step 3.2 — Load the data

Type this in the first cell and press Shift + Enter:

```
import pandas as pd import numpy as np import matplotlib.pyplot as plt import sqlite3, sys sys.path.insert(0, '..') from config.settings import DB_PATH conn = sqlite3.connect(str(DB_PATH)) df = pd.read_sql(""" SELECT gl.*, p.full_name FROM game_logs gl JOIN players p ON gl.player_id = p.player_id WHERE gl.min &gt; 0 """, conn) conn.close() df['game_date'] = pd.to_datetime(df['game_date']) print(f"Loaded {len(df):,} game logs") print(f"Unique players: {df['player_id'].nunique()}") df.head()
```

## Step 3.3 — Understand stat distributions

New cell:

```
for stat in ['pts', 'reb', 'ast', 'fg3m', 'min', 'pra']: print(f"{stat.upper():&gt;5}: mean={df[stat].mean():.1f}, " f"median={df[stat].median():.1f}, " f"std={df[stat].std():.1f}")
```

Most stats are right-skewed (mean > median). A few stars put up huge numbers while most players score modestly. This means 'average' can be misleading for props.

## Step 3.4 — Filter to prop-relevant players

```
season_avgs = df.groupby(['player_id', 'full_name', 'season']).agg( games=('pts', 'count'), avg_min=('min', 'mean'), avg_pts=('pts', 'mean'), avg_reb=('reb', 'mean'), avg_ast=('ast', 'mean'), avg_pra=('pra', 'mean'), ).reset_index() prop_players = season_avgs[(season_avgs['avg_min'] &gt;= 20) &amp; (season_avgs['games'] &gt;= 20)] print(f"Prop-relevant player-seasons: {len(prop_players)}") print(prop_players.sort_values('avg_pra', ascending=False).head(15).to_string(index=False))
```

## Step 3.5 — The hit rate question (KEY for props)

Props are not about averages — they are about **consistency**. A player averaging 8 rebounds might hit 8+ in 80% of games (great prop) or 50% of games (bad prop).

```
name = 'Nikola Jokic' latest = df['season'].max() jokic = df[(df['full_name'] == name) &amp;amp; (df['season'] == latest)] if not jokic.empty: print(f"{name} - {latest} - Rebounds") for line in [8.5, 9.5, 10.5, 11.5, 12.5, 13.5]: hr = (jokic['reb'] &amp;gt;= line).mean() print(f" Over {line}: {hr:.0%}")
```

**This is what your model will learn to do automatically.** Instead of you manually checking StatMuse, the model calculates this from features.

## Step 3.6 — Home/Away and Back-to-Back effects

```
df_sorted = df.sort_values(['player_id', 'game_date']) df_sorted['prev_game_date'] = df_sorted.groupby('player_id')['game_date'].shift(1) df_sorted['days_rest'] = (df_sorted['game_date'] - df_sorted['prev_game_date']).dt.days df_sorted['is_b2b'] = (df_sorted['days_rest'] == 1).astype(int) latest = df['season'].max() b2b_data = df_sorted[(df_sorted['season'] == latest) &amp;amp; (df_sorted['min'] &amp;gt;= 20)] for val, label in [(0, 'Normal rest'), (1, 'Back-to-back')]: s = b2b_data[b2b_data['is_b2b'] == val] print(f"{label}: PTS={s['pts'].mean():.1f}, REB={s['reb'].mean():.1f}, " f"MIN={s['min'].mean():.1f} (n={len(s):,})")
```

## Step 3.7 — Save and commit

Press Cmd + S in the notebook. Then in a separate Terminal tab:

```
cd ~/projects/nba-edge-model git add notebooks/ git commit -m "phase 1: EDA notebook complete" git push
```

## CHECKPOINT

- [ ] You can run the notebook without errors
- [ ] You see that variance matters more than average for props
- [ ] You see that opponent sample sizes are small (2-4 games)
- [ ] You see that home/away and B2B have small but real effects
- [ ] Notebook is committed to Git

# PART 4: FEATURE ENGINEERING

**Goal:** Transform raw game logs into the inputs (features) that an ML model can learn from.

**Why this matters:** The model is only as good as the features you give it. A stat line from 3 months ago is not as important as last week's games. Feature engineering encodes all of this.

## Step 4.1 — Understand the concept

For each player on each game day, BEFORE that game happens, we compute features like:

- pts_L5: Average points in last 5 games

- reb_L10: Average rebounds in last 10 games

- ast_hr_7_5_L10: % of last 10 games with 7.5+ assists

- min_std_L10: How stable are minutes (low = stable role)

- pts_trend: Is scoring trending up or down?

- opp_pts_allowed_L15: How many points the opponent allows recently

- is_b2b: Back-to-back game?

- days_rest: Days since last game

**Critical rule:** Every feature must only use data AVAILABLE BEFORE the game. If you accidentally include the result of the game you are predicting, that is called **data leakage** and it makes your model look amazing in backtesting but worthless in real life. Our code uses shift(1) to enforce this.

## Step 4.2 — The code is already in place

The zip file you downloaded already has all the feature engineering code. Open each file in VS Code and read through it:

**src/features/player_features.py** — Rolling averages (L5, L10, L20), hit rate features (what % of recent games exceeded common lines), trend features (L5 vs L10), consistency features (standard deviation), minutes features.

**src/features/context_features.py** — Rest days, back-to-back flag, home/away, season game number, game month.

**src/features/team_features.py** — Opponent defensive quality (how many points/rebounds/assists they allow), team pace (estimated possessions per game). Computed from the game logs themselves so the pipeline is self-contained.

**src/features/pipeline.py** — Ties everything together into one command.

## Step 4.3 — Run the feature pipeline

```
cd ~/projects/nba-edge-model source venv/bin/activate python -m src.features.pipeline
```

This takes 1-5 minutes. When it finishes, you should see:

Final dataset: 150,000+ rows x 120+ columns

Saved to .../data/processed/features.parquet

### Step 4.4 — Commit

```
git add . git commit -m "phase 2: feature engineering complete" git push
```

## CHECKPOINT

- [ ] data/processed/features.parquet exists
- [ ] The file has 100,000+ rows and 100+ columns
- [ ] No errors during the pipeline run
- [ ] Changes pushed to GitHub

# PART 5: MODEL TRAINING

**Goal:** Train an ML model that predicts player stat lines from the features we just built.

## Step 5.1 — Understand what we are predicting

We are training **one model per stat type**. Each model predicts the numeric value (e.g., 'Jokic will score 26.3 points tonight'). Later, we compare that prediction to the prop line to find edges.

Stats we model: pts, reb, ast, fg3m, pra, reb_ast

**Why regression, not classification?** Predicting '25.3 points' is more useful than predicting 'over 24.5 points.' Lines change — your prediction does not need to retrain. You can evaluate against ANY line from ANY book.

**Why XGBoost?** It is the go-to algorithm for tabular data like ours. It builds decision trees that learn patterns like 'when L5 rebound average is high AND opponent allows lots of rebounds AND player is home... predicted rebounds = X.'

**Train/test split:** We train on 2019-2024 and test on 2024-25. This simulates real-world usage — you train on the past, predict the future. This is NOT random splitting (which would leak future data into training).

## Step 5.2 — The model code is already in place

Open src/models/player_props.py in VS Code and read through it. Key things:

- get_feature_columns() automatically selects all feature columns

- FEATURES_TO_EXCLUDE lists raw stats (targets) and identifiers — these cannot be features

- The model uses early stopping to prevent overfitting

- We compare against a naive baseline (just predict L10 average)

- Feature importance shows which features matter most

## Step 5.3 — Train the models

```
python -m src.models.player_props
```

This takes 2-10 minutes. For each stat you will see:

- Train MAE and Test MAE (lower is better)

- Comparison against the L10 baseline

- Top 10 most important features

**What to expect:** The model should beat the L10 baseline by 3-10%. That might sound small, but in betting markets, a few percent edge is significant.

## Step 5.4 — Run the backtest

The edge finder and backtester are at src/models/edge_finder.py:

```
python -m src.models.edge_finder
```

This generates a report showing how the model would have performed on the 2024-25 season. The key question: **when the model says 70% chance of going over, does it actually hit around 70% of the time?**

At each threshold (55%, 60%, 65%, 70%, 75%), you will see how many picks qualified and what % actually hit. You will also see sample picks with predicted vs actual results.

### Step 5.5 — Commit

```
git add . git commit -m "phase 2: model training and backtesting complete" git push
```

## CHECKPOINT

- [ ] Models trained for all 6 stat types
- [ ] Each model beats the L10 baseline (even by a few %)
- [ ] Backtest report shows hit rates at various confidence thresholds
- [ ] Model files saved in data/processed/models/
- [ ] Everything committed and pushed

# PART 6: EDGE FINDING AND BACKTESTING

**Goal:** Deeper analysis — understand calibration, simulate ROI, identify strengths and weaknesses.

## Step 6.1 — Open a new notebook

Start Jupyter if not running:

```
jupyter notebook
```

Create a new notebook in notebooks/ called 03_model_evaluation.

## Step 6.2 — Load backtest results

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, sys sys.path.insert(0, '..')
from src.models.edge_finder import backtest_props stats = ['pts', 'reb', 'ast', 'fg3m',
'pra'] all_results = {} for stat in stats: try: all_results[stat] =
backtest_props(stat) print(f"{stat}: {len(all_results[stat]):,} predictions") except
Exception as e: print(f"{stat}: Error - {e}")
```

## Step 6.3 — Calibration plot

This is the most important chart for a betting model. It shows: when the model predicts 70% probability, does the player actually hit ~70% of the time?

```
fig, axes = plt.subplots(2, 3, figsize=(16, 10)) fig.suptitle('Model Calibration',
fontsize=14, fontweight='bold') for ax, stat in zip(axes.flatten(), stats +
['overall']): if stat == 'overall': bt = pd.concat(all_results.values(),
ignore_index=True) elif stat not in all_results: continue else: bt = all_results[stat]
bins = [0, 0.3, 0.4, 0.5, 0.55, 0.6, 0.65, 0.7, 0.8, 1.0] bt['prob_bin'] =
pd.cut(bt['prob_over'], bins=bins) cal = bt.groupby('prob_bin', observed=True).agg(
predicted=('prob_over','mean'), actual=('hit_over','mean'),
count=('hit_over','count')).dropna() ax.plot([0,1],[0,1],'k--',alpha=0.5)
ax.scatter(cal['predicted'], cal['actual'], s=cal['count']*2, alpha=0.7)
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual') ax.set_title(stat.upper());
ax.set_xlim(0,1); ax.set_ylim(0,1) plt.tight_layout()
plt.savefig('../data/processed/calibration_plot.png', dpi=150) plt.show()
```

## Step 6.4 — Simulated ROI

```
MULTIPLIER = 1.91 # Standard -110 odds THRESHOLD = 0.60 print(f"{'Stat':&amp;lt;8}
{'Picks':&amp;gt;6} {'Hits':&amp;gt;6} {'Hit%':&amp;gt;7} {'Profit':&amp;gt;8} {'ROI':&amp;gt;7}")
print("-" * 45) for stat, bt in all_results.items(): picks = bt[bt['prob_over'] &amp;gt;=
THRESHOLD] if len(picks) == 0: continue hits = picks['hit_over'].sum() total =
len(picks) hit_pct = hits / total profit = hits * (MULTIPLIER - 1) - (total - hits) * 1
roi = profit / total * 100 print(f"{stat.upper():&amp;lt;8} {total:&amp;gt;6} {int(hits):&amp;gt;6}
{hit_pct:&amp;gt;7.1%} ${profit:&amp;gt;7.1f} {roi:&amp;gt;6.1f}%")
```

## Step 6.5 — How to read the results

**Calibration plot:**

- Points close to the diagonal = well-calibrated (good)

- Points above the diagonal = model is underconfident (leaving edge on the table)

- Points below the diagonal = model is overconfident (losing money)

**ROI simulation:**

- Positive ROI at threshold 0.60+ = the model finds real edges

- Negative ROI = the model is not sharp enough yet (we can improve it)

- Even breakeven at this stage is fine — we have not done hyperparameter tuning or calibration yet

## Step 6.6 — Final commit

```
git add . git commit -m "phase 2 complete: models trained, backtested, evaluated" git push
```

## CHECKPOINT

- [ ] 6 trained XGBoost models (one per stat type)
- [ ] Each model beats the L10 naive baseline
- [ ] Calibration plots generated and interpretable
- [ ] Backtest report showing hit rates at various confidence thresholds
- [ ] ROI simulation complete (even if negative — honest evaluation is the point)
- [ ] Full project committed to GitHub with clean commit history
- [ ] You understand: features > model > prediction > probability > edge

# WHAT YOU NOW HAVE

Your complete model stack:

Game logs (5+ seasons)

|

Feature engineering (120+ features)

|

XGBoost models (6 stat types)

|

Predicted stat values

|

P(over line) via normal CDF

|

Edge = P(model) - P(implied)

|

Pick if edge > threshold

**WHAT COMES NEXT (Phase 3+)**

When you are ready, come back and we will build:

1. Probability calibration — Platt scaling to fix over/underconfidence

2. Game-level model — Predicted pace, spread, blowout risk feed back into props

3. Live odds integration — Pull actual Sleeper/sportsbook lines via API

4. Daily pipeline — One command to generate today's picks

5. Streamlit dashboard — Visual interface showing picks, confidence, and tracking

6. Hyperparameter tuning — Squeeze more accuracy from the models

Each of these is a 1-2 session build. The hard infrastructure work is done.